

Correction des exercices sur la vérification déductive

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 7 novembre 2025

Objectifs

- Comprendre les enjeux de la tâche de vérification des logiciels : nécessité de vérifier, démarche de vérification, complexité de la vérification.
- Comprendre naïvement les solutions de test et de test exhaustif.
- Faire la différence entre test et vérification.

L'exercice porte sur le programme C de la figure 1, qui implémente un algorithme de recherche dichotomique de la position d'un entier x dans un tableau t de n entiers, trié dans l'ordre croissant. L'intervalle des entiers entre n et m inclus est noté $n..m$. Les indices du tableau t valides sont dans l'intervalle $0..n-1$.

```
1 int binary_search(int t[], int n, int x) {  
2   int L = -1, R = n-1, m;  
3  
4   while(L  $\neq$  R) {  
5     m = (L+R)/2;  
6     if (t[m] > x)  
7       R = m-1;  
8     else  
9       L = m;  
10  }  
11  return L;  
12 }
```

FIGURE 1 – Recherche dichotomique dans un tableau trié

1 Exercice

1. Quel résultat ce programme va-t-il retourner pour $n = 5$, $x = 5$ et le tableau t suivant ?

i	0	1	2	3	4
$t[i]$	2	4	7	9	9

Le programme retourne 1. L'exécution est représentée par le tableau suivant :

Actions	L	R	m
$L := -1; R = n-1;$	-1	4	?
$L \neq R : m = (L+R)/2;$	-1	4	1
$t[m] \leq x : L = m;$	1	4	1
$L \neq R : m = (L+R)/2;$	1	4	2
$t[m] > x : R = m-1;$	1	1	2
$L = R : \text{return } L;$	1	1	2

La colonne *Actions* est de la forme $c : a$ où c est la condition qui est satisfaite et a est la séquence d'actions exécutées. Les valeurs obtenues pour la variable x sont dans la colonne de nom x .

2. Quel résultat ce programme va-t-il retourner pour $n = 5$, $x = 7$ et le même tableau t ?

Le programme retourne 2. L'exécution est représentée par le tableau suivant :

Actions	L	R	m
$L := -1; R = n-1;$	-1	4	?
$L \neq R : m = (L+R)/2;$	-1	4	1
$t[m] \leq x : L = m;$	1	4	1
$L \neq R : m = (L+R)/2;$	1	4	2
$t[m] \leq x : L = m;$	2	4	2
$L \neq R : m = (L+R)/2;$	2	4	3
$x < t[m] : R = m-1;$	2	2	3
$L = R : \text{return } L;$	2	2	3

3. A votre avis cet algorithme est-il juste ? Pourquoi ?

L'algorithme est faux pour plusieurs raisons :

- (a) il ne termine pas toujours,
- (b) il peut désigner $t[-1]$ qui n'existe pas et provoquer une erreur à l'exécution.

Cependant, les deux tests précédents n'ont pas révélé ces problèmes !

4. Exécuter le programme pour $n = 5$ et le même tableau t , mais pour $x=9$. Le programme est-il correct ? Si ce n'est pas le cas, quel(s) défaut(s) avez-vous trouvé ?

Avec $n = 5$ et le même tableau t , le programme ne termine pas pour $x = 9$.

5. Proposer une version correcte de ce programme.

Le programme est corrigé dans la figure 2. Ainsi corrigé, il termine toujours et calcule une position p tel que $p+1$ est la position d'insertion de x dans $t[0..n-1]$.

6. Si un tel programme était intégré au système de freinage ABS de votre voiture, auriez-vous confiance en sa vérification à partir de quelques tests ? Si non, comment vous y prendriez-vous pour le vérifier ?

Non, je n'aurais pas confiance. Un seul test ne suffit pas. On peut envisager 3 solutions pour augmenter la confiance dans cet algorithme :

- (a) Faire un test exhaustif. Nous verrons à la question 7 que ce n'est pas réalisable.
- (b) Faire un ensemble de tests couvrant l'ensemble de l'espace des données (voir question 8).
- (c) Faire une preuve. C'est ce que nous apprendrons à faire dans la suite du cours.

```

1 int binary_search(int t[], int n, int x) {
2   int L = -1, R = n-1, m;
3
4   while(L ≠ R) {
5     m = (L+R+1)/2;
6     if (t[m] > x)
7       R = m-1;
8     else
9       L = m;
10  }
11  return L;
12 }

```

FIGURE 2 – Recherche dichotomique dans un tableau trié, version corrigée.

7. Si nous faisons la vérification par un test exhaustif pour le cas $n=5$ avec x quelconque et t quelconque, en supposant qu'un entier peut prendre 65536 valeurs différentes (entier 16 bits) et qu'une exécution prend 1 seconde, quel serait le temps nécessaire pour effectuer ce test exhaustif?

Nombre de cas de test en considérant tous les tableaux, y compris ceux qui ne sont pas triés :

Nombre de valeurs de x : environ 60 000. Nombre de tableaux = $60\,000^5$. Temps approximatif : $60\,000^6$ secondes = $6^6 \times 10^{24} = 216^2 \times 10^{24} = 46656 \times 10^{24}$ secondes.

Sachant qu'il y a environ $3,2 \cdot 10^7$ secondes dans une année, ce temps serait $4,6 \cdot 10^{28} / 3,2 \cdot 10^7$, soit environ $1,43 \cdot 10^{21}$ années!

Le nombre de tableaux triés est de l'ordre de $60\,000^5 / 5!$. Le temps pour tester avec tous les tableaux triés est environ $1,43 \cdot 10^{21} / 120$, soit environ $1,2 \cdot 10^{19}$ années!

Il est donc clair que le test exhaustif n'est pas réalisable.

8. Puisque le test exhaustif n'est pas possible, comment vérifier si ce programme est juste (ou correct) ?

Etant donnée la réponse à la question 7, seules les stratégies 6b et 6c sont envisageables. Dans la stratégie 6b, couvrir l'ensemble de l'espace de données signifie choisir au moins un représentant de chaque classe de données jugées équivalentes entre elles. Pour faire du test fonctionnel, il faut déterminer un ensemble de critères qui rendent les données différentes. Sur l'exemple, ces critères sont :

- le fait que x soit ou non une valeur de t
- le fait que x soit en une seule occurrence ou non
- la position de x dans t
- l'existence de valeur ou non dans t .

Pour les tests structurels, on se base sur des critères de couverture du programme. En général, on veut couvrir toutes les instructions et tous les chemins d'exécution. Le problème est la couverture des itérations où le nombre de chemins peut être infini et très grand. Alors, pour les itérations, on se contente des cas 0, 1 et k itérations avec k assez grand.

Sur l'exemple, pour couvrir l'ensemble de données, nous pourrions faire les tests suivants :

a) un ensemble de tests fonctionnels, appelé test « boîte noire » suivant :

2 cas $x \in t$ et (x est en 1 seule occurrence, x est en plusieurs occurrences)

3 cas où $x \in t$ et ($x = t[1]$, $x = t[n]$, $x \neq t[1]$ et $x \neq t[n]$)

3 cas où $x \notin t$ et ($t[1] < x \wedge x < t[n]$, $x < t[1]$, $x > t[n]$)

1 cas où $n = 0$.

b) un ensemble de tests structurels appelés test "boîte blanche" suivant :

2 cas : 0 itération (ex : t vide avec $n=0$), 1 itération (ex : $n=1$, $x=t[1]$)

cas à plusieurs itérations : n grand

2 cas pour la conditionnelle : l'exécution de $L = m$ a lieu, l'exécution de $R = m-1$ a lieu.

9. Mais qu'est-ce qu'un programme "correct"? De quoi doit-on disposer pour pouvoir parler précisément de correction d'un programme? Est-ce que correction est synonyme de "conformité"?

Un programme est correct (1) s'il termine et (2) s'il calcule le résultat défini dans la spécification (son contrat). C'est-à-dire si, partant de données vérifiant la précondition, il aboutit à un résultat vérifiant la postcondition.

On ne peut donc parler de programme *correct* que si l'on dispose d'une spécification définissant le problème à résoudre et que l'on confronte le programme à sa spécification afin de vérifier que le résultat produit par le programme est *conforme* au résultat attendu défini dans la spécification.

10. Quel bilan peut-on faire concernant la vérification de programmes à partir de cet exercice?

Cette question était mal posée. Nous répondons ici à la question suivante : Comment doit-on modifier ce programme pour le rendre correct ?

Pour que le programme soit correct il faut remplacer le calcul de m par $m := (L+R+1)/2$. Avec ce calcul de m , il est possible de démontrer que l'programme termine toujours et en ayant calculé le "bon" résultat.

Noter que la non terminaison était due à l'instruction $m = (L+R)/2$ car, lorsque $L+1=R$ (intervalle de longueur 1), l'expression $(L+R)/2$ est égale à $(L+L+1)/2$, donc égale à L à cause de l'arrondi. Dans ce cas, si le programme exécute l'affectation $L = m$, alors L n'est pas modifié et cela continue ainsi infiniment.

Par contre, avec l'instruction $m = (L+R+1)/2$, l'expression $(L+R+1)/2$ est égale à $(L+L+1+1)/2$, donc égale à $L+1$ à cause de l'arrondi. Dans ce cas, si le programme exécute l'affectation $L = m$, alors L augmente de 1. Dans l'autre cas, l'expression $(L+R+1)/2$ est égale à $(R-1+R+1)/2$, donc égale à R . Dans ce cas, si le programme exécute l'affectation $R = m-1$ alors R diminue de 1. Donc, avec cette modification, l'intervalle $R-L$ décroît strictement et la condition $L = R$ finit par arriver et le programme termine toujours.